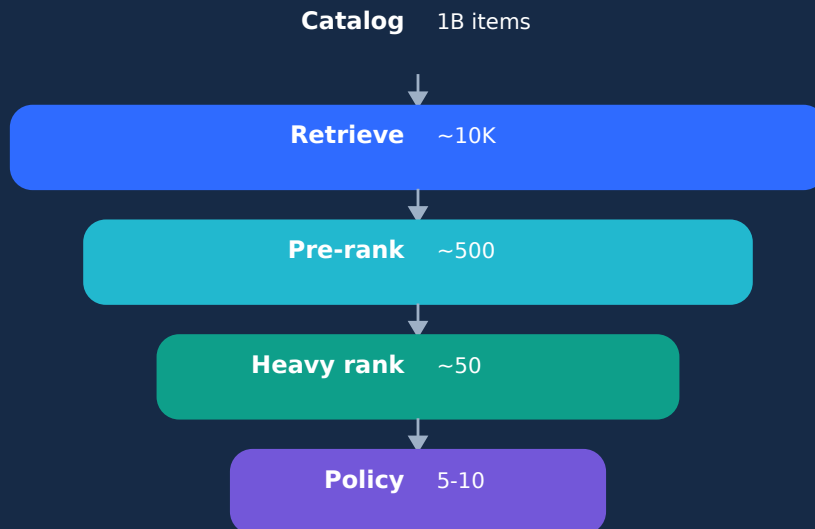


DAY 01 / 100

Design a Production-Grade Video Recommendation System

Machine Learning System Design - Consolidated Public Edition



A hierarchical decision system that retrieves, predicts, ranks, constrains, explores, and continuously learns.

[PUBLIC REVISION NOTES](#) | [HOME](#) + [UP NEXT](#) | [DESIGN EXERCISE](#)

What problem are we actually solving?

Given a user, a recommendation surface, the current session, and an eligible video catalog, return an ordered slate that maximizes durable user value under latency, safety, diversity, freshness, and business constraints.

The main architectural insight: a production recommender is not one model. It is a cascade that allocates increasingly expensive computation to progressively smaller candidate sets.

Two surfaces, two intents

- **Home:** broad discovery from long-term interests, recent sessions, subscriptions, popularity, and exploration.
- **Up Next:** strong conditioning on the current video, recent sequence, session continuation, and avoidance of near-duplicates.
- Use shared infrastructure, but separate objectives, features, candidate budgets, and experiments where behavior differs.

Contents

- 01 Scope and requirements
- 02 Architecture and request flow
- 03 Retrieval and ANN
- 04 Ranking and value modeling
- 05 Data, bias, and learning
- 06 Serving, reliability, and monitoring
- 07 Roadmap, trade-offs, and checklist

Design stance

Optimize satisfaction and long-term value, not a single proxy such as click-through rate. Use the simplest model that meets measurable quality and operational goals. Treat every scale number and latency allocation below as an explicit design assumption, not a claim about a named company's internal system.

Requirements, assumptions, and success criteria

Dimension	Public design assumption	Why it matters
Catalog	Up to 1B active videos; eligibility varies by market, age, policy, and rights.	Forces multi-stage retrieval, sharding, compression, and aggressive filtering.
Response	5-20 ranked videos per page; pagination and refresh are supported.	The final object is a slate, not independent scores.
Latency	Illustrative P99 budget: 250 ms for the recommendation backend.	Requires parallel fan-out, bounded stages, caching, and fallbacks.
Freshness	Session signals in seconds; trends in minutes; long-term features in hours.	Different features need different pipelines and serving stores.
Availability	Recommendation failure must not break video playback or the home shell.	Recommendations are a soft-state dependency with graceful degradation.

Functional

- Personalize Home and Up Next.
- Blend long-term preference with short-term intent.
- Handle new users, videos, and creators.
- Support diversity, novelty, freshness, exploration, and safety.
- Learn from impressions, watches, skips, explicit feedback, and returns.

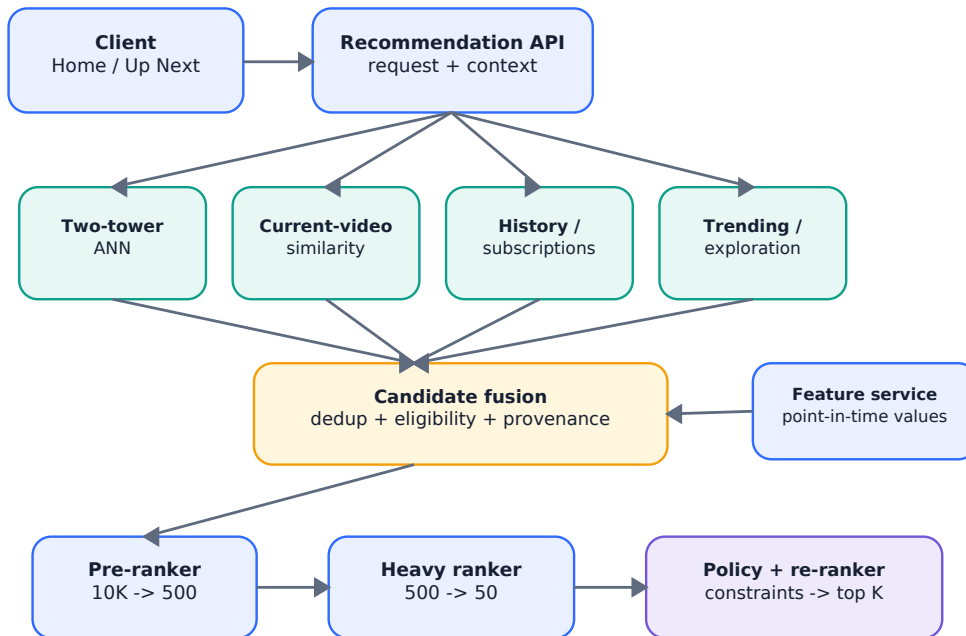
Non-functional

- Predictable P95/P99 latency and horizontal scale.
- High availability and bounded dependency fan-out.
- Reproducible training with point-in-time feature correctness.
- Cost-efficient inference, observability, privacy, and rollback.
- Regional availability and policy enforcement before response.

Primary product goal: increase satisfied viewing and durable retention. Guardrails: negative feedback, safety, creator concentration, latency, error rate, and inference cost.

End-to-end online recommendation path

The online path separates retrieval, feature assembly, ranking, and policy so that each stage can scale, fail, and evolve independently.



ONLINE PATH

Figure 1. Independent retrievers execute in parallel. Candidate provenance and scores survive fusion and become ranking features.

Boundary	Contract	Failure behavior
Recommendation API	User/session/surface/context -> request ID, deadlines, and top K.	Reject malformed input; use anonymous/contextual path when identity is unavailable.
Retrieval	Per-channel candidate IDs, scores, source, and index/model version.	Timeout one channel without waiting for all; preserve minimum candidate floor.
Ranking	Candidates + point-in-time features -> calibrated outcome predictions.	Use smaller model or cached scores when the heavy model misses its deadline.
Policy	Predictions + constraints -> valid, diverse, safe slate.	Fail closed on safety/rights; fall back on non-personalized safe inventory.

Request lifecycle and component responsibilities

Step	Action	Key implementation detail
1	Authenticate and normalize	Resolve anonymous/user ID, locale, surface, device, network, age/market policy, and request deadline.
2	Assemble context	Fetch long-term profile, recent session state, current video, recent exposures, and experiment assignments.
3	Retrieve in parallel	Query ANN, co-watch/current-video, subscriptions, history, trending, and exploration channels.
4	Fuse and filter	Deduplicate; enforce availability, rights, safety, block lists, and recently-consumed policy; retain provenance.
5	Pre-rank	Use cheap features/model to reduce roughly 10K candidates to hundreds.
6	Heavy rank	Predict click, conditional watch time, completion, satisfaction, negative feedback, and return/session value.
7	Policy and re-rank	Combine predictions, normalize objectives, enforce slate constraints, diversify, and reserve exploration slots.
8	Respond and log	Return IDs plus tracking tokens; atomically log model/index versions, positions, scores, and propensities.

Hard filters before ranking

Deleted/private videos, geo/rights restrictions, age gates, safety policy, user blocks, and unavailable media must not enter the eligible set.

Soft constraints after ranking

Topic and creator caps, freshness, long-tail exposure, novelty, exploration, and repeated-content penalties shape the slate.

Idempotency and traceability

Every request carries a stable request_id. Impression events include request_id, video_id, position, surface, policy/model/index versions, experiment IDs, and a timestamp. Engagement events join back to the exact exposure rather than to a later reconstructed feature state.

Multi-channel candidate generation

Retrieval optimizes recall under extreme catalog scale. No downstream ranker can recover a relevant video that never entered the candidate pool.

Retriever	Signal captured	Best use	Main risk
Two-tower ANN	User/session-to-video embedding affinity	Massive-scale personalized retrieval	Embedding/index staleness and popularity-skewed negatives
Current-video	Co-watch, semantic, creator, or topic similarity	Up Next and session continuation	Near-duplicate loops
Content-based	Title, transcript, audio/visual, tags, topic	New-video retrieval and semantic coverage	Metadata manipulation or weak content understanding
History-based	Recent watches, searches, skips, and topic transitions	Short-term intent	Overreacts to accidental activity
Subscriptions	Explicit creator/series preference	High-precision inventory	Narrow coverage
Trending	Momentum by region, language, and cohort	Freshness and fallback	Popularity reinforcement
Exploration	Uncertain, new, or underexposed content	Learning and cold start	Short-term quality cost

Budgeting and fusion

Allocate per-channel quotas dynamically by surface, user state, and channel health. Over-fetch from high-recall channels, deduplicate by video_id, then cap any single source so it cannot collapse candidate diversity. Preserve raw scores and source flags; never compare uncalibrated scores from unrelated retrievers as if they shared a scale.

Retrieval metric hierarchy: per-channel Recall@K -> union recall -> unique coverage -> eligible coverage -> downstream contribution and incremental online lift.

Two-tower retrieval and contrastive learning

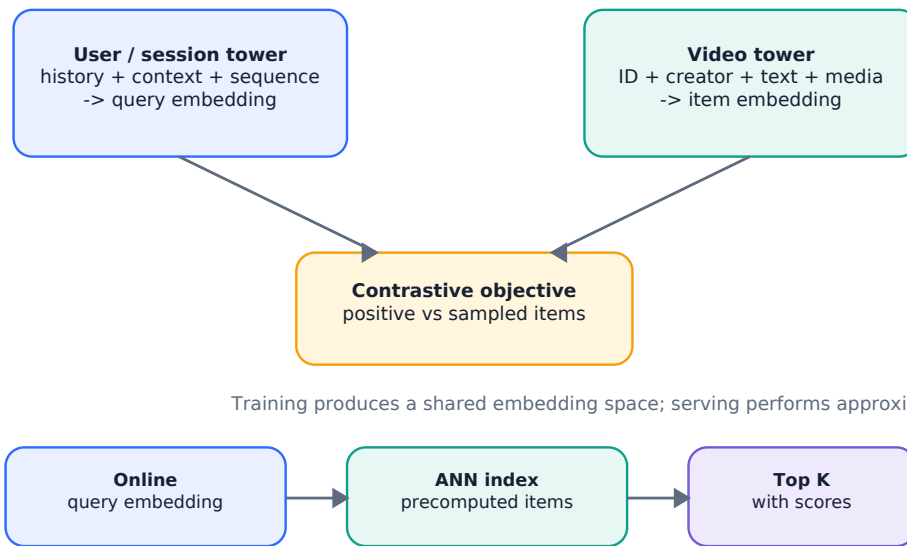


Figure 2. Video embeddings are precomputed; the online tower produces a query embedding from the user and current session.

Core scoring model

$\text{score}(u, v) = q(u, \text{session}, \text{context}) \cdot d(v)$. Train positives from qualified engagements, not clicks alone. Use sampled softmax or a contrastive loss to place engaged items near the query while separating negatives. The classic two-stage YouTube architecture motivates separating candidate generation from ranking [1].

Query tower

- Long-term history and topic/creator affinity.
- Recent ordered session actions and current video.
- Surface, locale, device, time, and network.
- Anonymous/cohort pathway when history is absent.

Item tower

- Video and creator IDs with freshness handling.
- Title, transcript, tags, topic, duration, and language.
- Audio/visual or multimodal embeddings when justified.
- Safety/quality metadata for eligible representation.

Important correction: in-batch negatives are efficient but not unbiased. Popular items are sampled more often, and another user's positive can be a false negative. Frequency correction, masking known positives, and mixed negative sources reduce damage [3].

ANN indexing, sharding, and freshness

Choice	When it fits	Trade-off
HNSW	High-recall, low-latency search for in-memory shards or hot inventory.	Memory-heavy; updates/deletes and billion-scale replication require care.
IVF + PQ	Very large catalogs where compression and bounded probes matter.	Lower memory, but quantization and coarse partitioning reduce recall.
ScaNN-style search	Accelerated maximum inner-product search with tuning for recall/latency.	Operational choice depends on runtime and hardware ecosystem.
Exact search	Small filtered catalogs, offline evaluation, or recall oracle.	Cost grows linearly; unsuitable as the only online search over 1B vectors.

Partition before searching

Filter or route by hard eligibility dimensions such as market, language, policy class, and active status. Then shard the remaining vector space by consistent hash, coarse centroid, or inventory tier. Fan out only to bounded shards. Merge shard-local top K results and attach the index version used.

Index lifecycle

Path	Mechanism	Correctness requirement
New video	Nearline embed -> delta index -> periodic base rebuild	Available quickly without waiting for the full index build.
Updated metadata	Re-embed when representation-relevant fields change	Version feature schema and embedding model together.
Delete / policy block	Immediate deny-list or tombstone plus asynchronous physical removal	Hard filters must override stale index membership.
Rebuild	Shadow build -> recall/latency validation -> atomic alias swap	Old version remains available for rollback.

At billion scale, vector compression and memory layout are first-class design decisions; FAISS research demonstrates GPU-based billion-scale similarity search and the underlying speed/accuracy/memory trade-off [8].

Candidate fusion, eligibility, and provenance

A fusion service converts heterogeneous channel outputs into a consistent candidate record.

Candidate record

video_id, source_flags, per_source_rank, per_source_score, retrieval_timestamp, index_version, duplicate_group, eligibility_flags, recent_exposure_count, and optional exploration_probability

Operation	Why it exists	Common mistake
Deduplicate	The same video may arrive from many channels.	Dropping source evidence instead of aggregating provenance.
Eligibility filter	Rights, safety, age, privacy, and availability are hard constraints.	Assuming the ranker will learn to suppress forbidden items.
Consumption policy	Avoid immediate repeats while allowing rewatch when appropriate.	Treating all historical watches as permanent exclusions.
Source normalization	Scores from ANN, trends, and subscriptions have different semantics.	Sorting raw cross-channel scores directly.
Quota control	Guarantee minimum coverage and cap dominant retrievers.	Static quotas that ignore user state or channel health.
Pre-ranking	Spend cheap compute to reduce 10K to hundreds.	Using expensive cross-features before the candidate set is small.

A candidate found independently by several channels may be stronger, but channel count is only a feature. It is not a substitute for learned relevance or calibrated source reliability.

Duplicate groups

Near-identical uploads, clips, language variants, episodes, and mirrored media require a semantic duplicate/group ID. Fusion can keep the best eligible representative; re-ranking can cap a group so the final slate does not waste positions on essentially the same content.

Cascaded ranking architecture

Ranking estimates candidate-level outcomes. The policy layer decides what to show after considering trade-offs across the entire slate.

Stage	Input size	Typical model	Goal
Pre-ranker	~10K -> ~500	GBDT, linear/wide model, or small MLP	Cheaply remove obvious low-value candidates while preserving recall.
Heavy ranker	~500 -> ~50	Deep cross-feature or multi-task network	Predict multiple engagement and satisfaction outcomes.
Policy / value	~50	Calibrated weighted utility, constrained optimizer, or learned value function	Translate predictions into comparable user value.
Slate re-ranker	~50 -> 5-20	Greedy/beam/optimization/learned slate model	Enforce diversity, freshness, safety, exploration, and caps.

Multi-task predictions

- P(click)
- E(watch_time | click)
- P(completion)
- P(like/share)
- P(session continuation)
- P(negative feedback)
- P(satisfaction survey positive)
- P(return within horizon)
- Expected abandonment risk
- Quality/safety risk score

Multi-task ranking is especially useful when objectives have different label frequencies and relationships. Shared experts with task-specific gates, such as MMoE, are one option; monitor negative transfer rather than assuming joint training always helps [2][4].

Do not optimize only CTR. A clickbait video can win clicks while losing watch quality, satisfaction, or long-term trust.

From predictions to a value function

Expected watch contribution

$$\text{ExpectedWatch} = P(\text{click}) * E(\text{watch_time} \mid \text{click})$$

A lower-CTR video can still be better when conditional watch time and satisfaction are much higher. Likewise, a long video should not receive an automatic advantage merely because its raw watch-time ceiling is larger; use normalization, capped utility, percent completion, and explicit satisfaction signals.

Conceptual utility

$$\text{Utility} = w1*\text{Click} + w2*\text{NormalizedWatch} + w3*\text{Completion} + w4*\text{Satisfaction} + w5*\text{Return} - w6*\text{NegativeFeedback}$$

Requirement	Reason
Calibration	A predicted 0.8 should correspond to about 80% observed frequency in the relevant slice before probabilities are combined.
Comparable units	Minutes, probabilities, and survey scores cannot be added meaningfully without normalization or a learned transformation.
Segment checks	Global calibration can hide failures by country, device, video duration, newness, or traffic source.
Constraint separation	Hard safety/rights rules should not be traded away by a high utility score.
Long-term validation	Short-term watch gains may reduce satisfaction or retention; experiment over appropriate horizons.

Prediction and decision are separate: models estimate outcomes; a versioned policy converts those estimates into a slate under explicit constraints.

Feature system and session intent

Feature family	Examples	Freshness
User	Long-term topics, creator affinity, language, membership age, embedding	Hours / nearline
Session	Ordered watches, skips, searches, dwell, topic transitions, current video	Seconds
Video	Topic, creator, duration, language, freshness, quality, popularity	Minutes to daily
User x video	Embedding similarity, creator/category affinity, prior exposure, novelty	Mixed
Context	Surface, device, locale, time, network, referrer	Request-time
Retrieval	Source flags, per-source ranks/scores, number of sources, index version	Request-time

Long-term representation

Stable preferences such as technology, music, sports, language, and creator affinity. Useful for Home discovery and sparse sessions.

Short-term intent

A sequence like LLM -> RAG -> vector database -> evaluation indicates current intent that may dominate the long-term profile.

Sequence encoder

GRU, temporal convolution, attention, or Transformer over recent events produces a session embedding. SASRec is a classic self-attentive sequential recommendation example [5].

Operational decision

Use a Transformer only when incremental quality justifies tail latency, memory, training cost, and failure surface. A smaller sequence model or summary features may win.

Feature parity rule: the training pipeline and online feature service must share definitions, schemas, and event-time semantics. Point-in-time joins prevent future information from leaking into training.

Learning-to-rank formulations and model choices

Formulation	What it learns	Strength	Risk / cost
Pointwise	Outcome for each candidate independently	Simple, scalable, strong baseline	Does not directly optimize relative order
Pairwise	A should rank above B	Closer to preference ordering	Pair construction and weighting matter
Listwise	Quality of the whole ordered list	Closer to final ranking objective	More complex training and serving assumptions
Slate-aware	Interactions among selected items	Can model redundancy and complementarity	Large action space and counterfactual difficulty

Practical progression

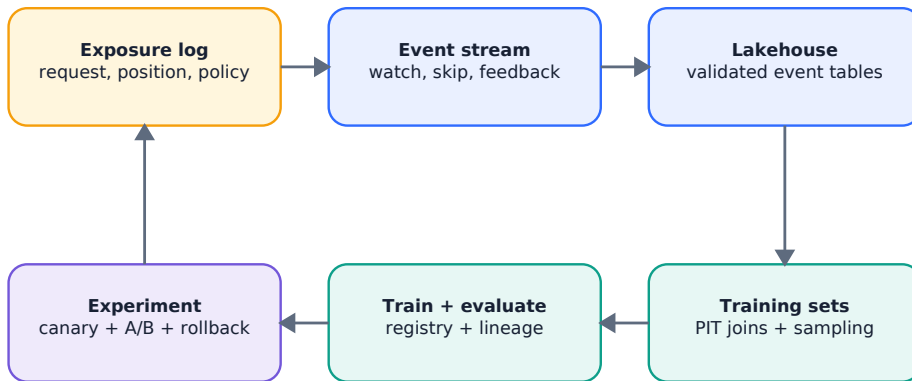
Phase	Model	Why
Baseline	Logistic/linear or GBDT ranker	Fast iteration, explainability, strong tabular baseline.
Scale-up	Wide-and-deep or MLP with embeddings	Combines memorized crosses with generalization.
Multi-objective	Shared-bottom, MMoE, or task-specific heads	Efficiently predicts sparse and dense outcomes together.
Session-aware	Sequence encoder feeding retrieval/ranking	Captures ordered short-term intent.
Advanced	Learned policy/slate objective	Useful only after reliable logging, evaluation, and constraints exist.

Loss weighting

Balance label sparsity, task importance, and gradient magnitude. Monitor each head's calibration and segment performance. Multi-task learning can cause negative transfer when task relationships change; task-specific gates or separate models may be safer.

Sophistication is not the goal. Choose the best quality / latency / cost / stability point and prove the gain through ablation plus online experiments.

Feedback events and the learning loop



Nearline features update session state; batch pipelines retrain and validate models.

Figure 3. Exact exposure logging connects online policy decisions to offline examples and experiment results.

Event	Minimum fields	Purpose
Impression	request_id, video_id, position, surface, timestamp, policy/model/index versions, propensity	Defines exposure; anchors all later labels.
Engagement	impression_id, click/play, watch duration, completion, skip, seek, timestamp	Dense behavioral labels and sequence state.
Satisfaction	like/dislike, not interested, report, survey response	Balances watch-time proxies.
Session	start/end, continuation, abandonment, return horizon	Longer-term outcomes.
Content	video/creator/version, metadata, transcript, quality, safety, rights	Item features and eligibility history.

Without impression and position data, an unclicked item is not a valid negative. It may have been below the fold, not rendered, or never meaningfully examined.

Training examples, labels, and negative sampling

Dataset construction

- Join engagement to the exact impression and event-time feature snapshot.
- Define label windows explicitly: click in seconds, watch/completion in-session, return over days.
- Remove bots, corrupted clients, duplicate events, and policy-invalid inventory.
- Use global temporal splits and user/item leakage checks; keep a final untouched test window.
- Version code, schemas, feature definitions, sampling policy, and source snapshots for reproducibility.

Negative	Value	Failure mode	Mitigation
Random catalog	Broad separation	Often too easy and may be ineligible	Sample from eligible inventory and mix with harder sources
In-batch	Efficient contrastive training	Popularity bias and false negatives	Frequency correction; mask known positives; larger diverse batches
Impression skip	Behaviorally plausible	Position/exposure bias; skip may not mean dislike	Use examination signals, dwell thresholds, and propensity correction
Hard negative	Fine-grained discrimination	Can overfocus on model errors or mislabeled positives	Mine from multiple retrievers; cap hardness; audit examples

Positive labels

Separate click, qualified watch, completion, explicit satisfaction, and return rather than collapsing everything into one binary label. Duration-conditioned watch thresholds help avoid labeling a 20-second watch identically for a 30-second clip and a 60-minute video.

Data quality is part of model quality. Monitor event completeness, join rate, delay, duplicate rate, label prevalence, and feature null rates before training.

The hardest ML problem: policy-generated feedback

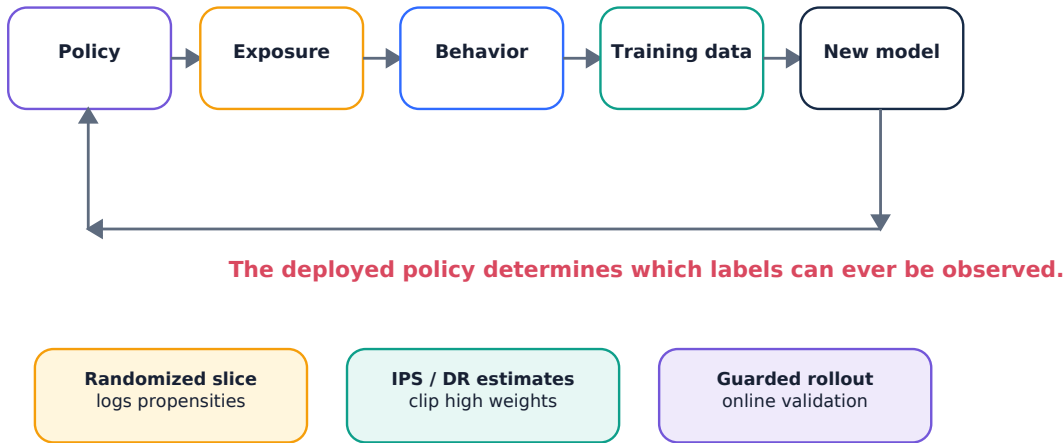


Figure 4. The system learns only from content selected by previous policies; this creates a closed feedback loop.

Bias	Mechanism	Observed symptom
Position	Higher positions are examined more often independent of relevance.	Top-ranked items appear better than they are.
Exposure	Unshown inventory has no behavioral label.	Model overfits the support of the old policy.
Selection	Old retrievers decide which candidates reach training logs.	New ranker cannot learn about missing regions of the catalog.
Popularity	More exposure creates more interactions and more training weight.	Head items dominate; new/long-tail items struggle.
Trust / presentation	Users interpret rank, thumbnail, or creator as evidence of quality.	Clicks mix preference with UI and policy effects.

Unbiased learning-to-rank formalizes propensity-weighted learning from biased feedback [6]. In production, estimated propensities, click noise, support mismatch, and high-variance weights must be measured and guarded.

Debiasing and counterfactual evaluation

Technique	What it needs	Benefit	Caution
Randomized traffic	Small, protected exploration slice	Creates support beyond the incumbent policy	User cost; safety and segment caps
Position model	Examination/click model or randomized swaps	Corrects part of position bias	Model misspecification and trust bias
IPS	Logged action propensity and overlap	Counterfactual reweighting	High variance; clip/self-normalize with bias trade-off
Doubly robust	Reward model plus propensities	Can reduce variance and remain robust to one model error	More implementation and validation complexity
Replay / off-policy eval	Logged actions with known probability	Screens policies before launch	Rejects unsupported actions; limited effective sample size
Online A/B test	Random assignment and guardrails	Measures actual causal product impact	Cost, duration, novelty, interference

Propensity logging contract

For each exploratory decision, log the candidate set, selected action/slate, probability under the behavior policy, context, and policy version. A deterministic production policy has zero probability for unchosen actions, so it cannot support general counterfactual claims without randomized coverage.

Practical gates

- Require minimum overlap and effective sample size before trusting off-policy estimates.
- Clip extreme weights; report sensitivity across clipping thresholds.
- Compare multiple estimators and validate them on known online experiments.
- Use counterfactual estimates to reduce risk, not to replace controlled online testing.

Contextual-bandit work demonstrates learning and offline replay from randomized logged interaction data [7].

Exploration and cold start

Exploitation

Choose items with high estimated value. Maximizes immediate expected reward but reinforces what the model already knows.

Exploration

Allocate controlled exposure to uncertain, new, or underexposed items. Generates information and improves future policy support.

Policy options

- Epsilon-greedy as a simple baseline.
- UCB for optimism under uncertainty.
- Thompson sampling for posterior sampling.
- Contextual bandits when choice depends on user/item context.
- Quota/reserved slots for operational control.

Cold-start case	Why collaborative signals fail	Mitigation
New user	No personal history	Locale/device/time, onboarding interests, contextual trending, anonymous session sequence, diverse exploration
New video	No interaction history	Text/transcript/audio/visual embeddings, creator priors, quality/safety checks, nearline indexing, controlled exposure
New creator	No creator-level reputation or affinity	Content understanding, policy checks, small exposure budget, quality-aware creator priors
New market	Sparse local interactions and language coverage	Cross-market transfer, language models, regional editorial/trending, local guardrails

New user, new video, and new creator are different ML problems. A single 'cold start' fallback hides different missing signals and risk levels.

Slate re-ranking, diversity, and constraints

Individual ranking can produce: RAG | RAG | RAG | RAG | RAG

A better slate may be: RAG | Vector DB | AI Agents | LLM Serving | Evaluation

Conceptual slate score

$$\text{SlateValue} = \text{sum}(\text{item utility}) + \lambda \cdot \text{diversity} + \gamma \cdot \text{novelty} + \eta \cdot \text{exploration} - \text{penalties}$$

Mechanism	Purpose	Implementation options
Topic/creator caps	Prevent concentration and repetition	Greedy constraint satisfaction or bounded optimization
Similarity penalty	Avoid near-duplicate videos	Embedding/metadata distance, duplicate groups, maximal marginal relevance
Freshness/novelty	Create discovery and adapt to new inventory	Bonuses with calibrated ceilings and segment controls
Exploration slot	Collect information under a known policy	Bandit score or quota with logged probability
Safety and rights	Never return forbidden inventory	Hard eligibility filter before and after ranking
Business/editorial	Honor explicit product obligations	Versioned transparent rules; measure opportunity cost

Avoid a common error

Diversity is not 'randomness.' It should preserve high expected user value while reducing redundancy across the slate. Evaluate both relevance loss and gains in discovery, satisfaction, and creator/topic concentration.

Re-run hard eligibility after any re-ranking or rule injection. Policy layers must not reintroduce content removed earlier.

Temporal splits and offline metrics

Training: Jan-Feb | Validation: Mar 1-15 | Test: Mar 16-31

Random user-event mixing can leak future interactions and produce optimistic results. Use event-time splits that reproduce the serving horizon. Freeze candidate availability and feature values as of each historical request.

Layer	Metrics	Question answered
Retrieval	Recall@K, HitRate@K, eligible/unique coverage, per-source contribution	Did a valuable item enter the candidate pool?
Ranking	NDCG@K, MRR, MAP where appropriate, per-task loss	Are useful candidates ordered early?
Calibration	Reliability plots, ECE/Brier by segment	Are probabilities meaningful for decisions?
Slate	Intra-list diversity, topic/creator concentration, novelty, freshness	Is the final list useful without being redundant?
System	P50/P95/P99 latency, error rate, throughput, cache hit, cost	Can the design serve reliably and economically?

Evaluation slices

- New vs established users, videos, and creators.
- Home vs Up Next; anonymous vs signed-in; short vs long sessions.
- Country, language, device, network quality, and video duration.
- Head vs torso vs long-tail inventory and policy-sensitive cohorts.

Offline improvement is necessary but insufficient. AUC or NDCG can rise while satisfaction, retention, diversity, or latency deteriorates online.

Online experiments and decision framework

Metric class	Examples	Decision use
Primary	Satisfied watch time, qualified plays, session value	The main success criterion for the surface.
Long-term	Return rate, retention, survey satisfaction	Detect proxy optimization and novelty effects.
Negative	Dislike, not interested, report, rapid abandonment	Ensure engagement is not bought through poor experiences.
Ecosystem	Creator/topic concentration, long-tail exposure, freshness	Protect supply diversity and discovery.
System guardrails	P99 latency, errors, availability, GPU/CPU cost	Block launches that are operationally unsafe.
Safety guardrails	Policy violations and sensitive-segment harm	Fail closed; may require human or specialist review.

Experiment mechanics

- Randomize at a stable unit that minimizes contamination, commonly user or account.
- Log exposure and assignment before outcomes; use pre-declared primary metrics and guardrails.
- Run an A/A test for instrumentation, then ramp 1% -> 5% -> 25% -> 50% with automated rollback thresholds.
- Account for novelty, weekday/seasonality, delayed retention, and repeated-measures variance.
- Report confidence intervals and practical effect size, not only statistical significance.

Launch rule: ship only when the primary metric improves or remains within an agreed non-inferiority bound, no hard guardrail regresses, and operational cost is acceptable.

Latency budget, parallelism, and deadlines

Illustrative P99 latency budget - 250 ms

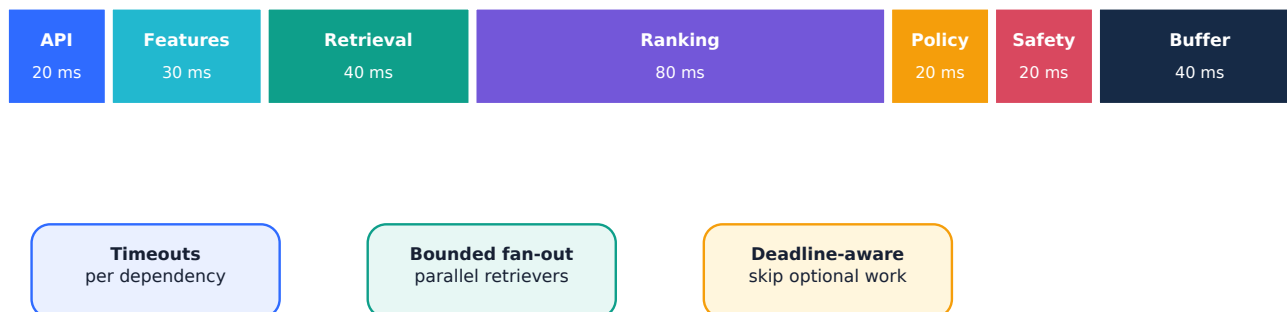


Figure 5. One illustrative P99 allocation; exact values are product-specific and must be measured.

Technique	Application
Parallel fan-out	Execute independent retrievers and feature fetches concurrently; total time approaches the slowest required branch.
Bounded concurrency	Cap shards/channels per request to prevent retry storms and tail amplification.
Deadline propagation	Each service receives the remaining deadline and stops optional work early.
Micro-batching	Batch candidate inference within a short window to improve accelerator utilization.
Model cascade	Spend heavy-model compute only after cheap retrieval and pre-ranking reduce the set.
Hedging	Use carefully for rare slow replicas; cancel losers and cap extra load.

Capacity planning

Derive peak recommendation QPS from DAU, requests per user, diurnal multiplier, and region. Separately estimate event-write QPS, ANN shard load, feature lookups, candidates scored per second, and accelerator batch capacity. Validate with load tests at peak plus failure headroom.

Feature freshness, caching, and cost

Signal	Typical freshness	Serving path
Current video and session	Real-time / seconds	Session store or request context
Trending	Seconds to minutes	Streaming aggregation + regional cache
Popularity	Minutes	Nearline aggregation + feature store
Long-term profile	Hours / nearline	Profile store with version and TTL
Video metadata	Minutes to daily	Metadata service with event-driven invalidation
Video embedding	Nearline plus batch rebuild	Delta ANN index + versioned base index

Good cache targets

- Regional/cohort trending sets.
- Immutable video metadata and embeddings.
- Short-TTL user embeddings.
- Expensive candidate sets plus real-time filtering.
- Model artifacts and feature schemas.

Cost controls

- ANN and vector compression.
- Quantization and smaller pre-rankers.
- Batch/vectorized inference.
- Candidate budget by user/surface value.
- CPU fallback and accelerator admission control.

Cache correctness

A cached candidate list is never the final answer. Reapply deletion, rights, safety, recent-consumption, and session filters at request time. Include locale, surface, policy version, and model/index version in cache keys where they affect validity.

Core trade-off: personalization versus cacheability. Cache reusable intermediate representations, then apply lightweight request-time personalization and filtering.

Graceful degradation and failure containment

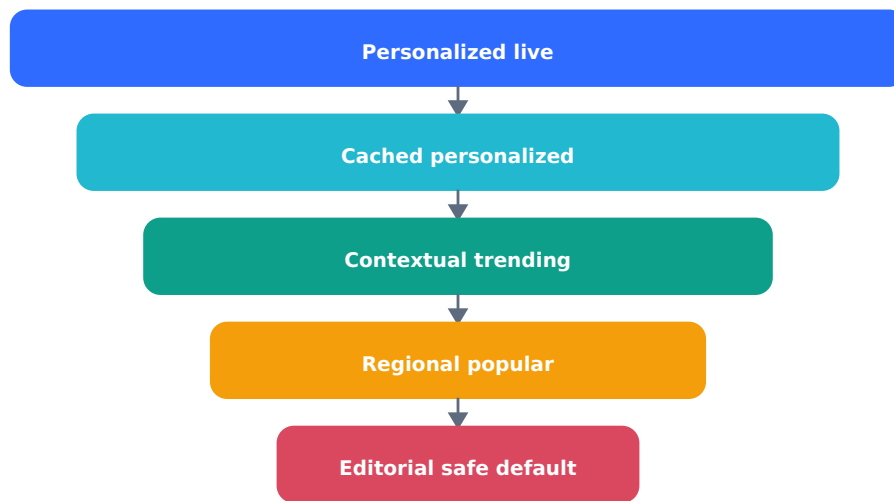


Figure 6. Quality should degrade stepwise while the core product remains available.

Failure	Detection	Fallback
One retriever times out	Per-channel deadline and candidate floor	Continue with healthy channels; rebalance quotas.
Feature store unavailable	Timeout/null-rate alarm	Use safe defaults, stale bounded features, or smaller context-free model.
Heavy ranker unhealthy	Latency/error/canary guardrail	Pre-ranker or last-known-good model.
ANN index bad	Recall sentinel, version health, empty-rate	Rollback alias or use cached/trending candidates.
Event pipeline delayed	Consumer lag and join-rate alarm	Serving continues; freeze affected nearline features; block retraining.
Policy/safety dependency fails	Hard dependency health	Fail closed to prevalidated safe inventory.

Operational controls

Use circuit breakers, load shedding, admission control, idempotent event writes, jittered retries, region-local caches, multi-zone replicas, model shadowing, and one-click rollback. Exercise fallbacks continuously; an untested fallback is not reliable.

Observability across system, data, ML, and product

Layer	Signals	Alert examples
System	QPS, P50/P95/P99, errors, saturation, GPU/CPU, memory	P99 breach; accelerator queue growth; regional error spike
Retrieval	Empty rate, Recall@K sentinel, source share, overlap, index age	One source dominates; new-video coverage collapses
Data	Schema, lag, duplicates, nulls, join rate, label prevalence	Impressions arrive but engagement joins fall
Features	Freshness, missingness, distribution drift, skew	Online/training skew or stale session state
Model	Score distribution, calibration, task loss, embedding norms/drift	Scores saturate; one task regresses; embedding collapse
Product	Watch quality, satisfaction, retention, diversity, concentration	Watch rises while negative feedback rises
Experiment	Assignment balance, exposure, lift, confidence, guardrails	Sample-ratio mismatch or inconsistent treatment exposure

Golden path and sentinel sets

Maintain deterministic test users/sessions and a curated item set to catch empty responses, banned-item leakage, index/version mismatch, sequence regressions, and excessive duplicates. Pair offline sentinels with sampled online traces and end-to-end synthetic requests.

Silent failure principle

A recommender can return HTTP 200 while quality collapses. Operational health and recommendation health require separate SLOs and dashboards.

Every alert should name an owner, severity, runbook, automated mitigation, and rollback target.

Safety, privacy, governance, and ML failure modes

Risk	Mitigation
Objective hacking	Use satisfaction and negative-feedback labels, long-horizon tests, caps, and qualitative audits.
Filter bubbles	Diversity constraints, exploration, topic coverage, user controls, and concentration monitoring.
Creator concentration	Exposure dashboards, caps/regularization, cold-start budgets, and fairness review.
Embedding/model drift	Freshness SLOs, drift detection, shadow evaluation, scheduled/triggered retraining.
Retrieval collapse	Per-source quotas, union-recall tests, source attribution, and index rollback.
Unsafe or unavailable content	Defense-in-depth eligibility before retrieval where possible and before response always.
Privacy leakage	Data minimization, retention limits, pseudonymous IDs, access control, deletion propagation.
Experiment harm	Small ramps, segment guardrails, kill switch, policy review, and explicit exclusion groups.

Model governance checklist

- Model card: objective, labels, data windows, segments, limitations, and intended surface.
- Lineage: code, feature schema, data snapshot, sampling policy, hyperparameters, and artifact hash.
- Approval: offline gates, safety/privacy review, shadow/canary evidence, and named owner.
- Deployment: immutable version, compatible feature/index versions, automated rollback, and audit trail.
- Deletion: user/content deletion must propagate to feature stores, training data policy, caches, and indexes.

Hard policy constraints are not ranking preferences. They must remain enforceable even when models, caches, or indexes are stale.

MVP to mature system roadmap

Phase	Build	Exit criteria
MVP	Trending + history/content retrieval; GBDT/small neural ranker; watch/click labels; caching; safe fallback; basic A/B testing	Reliable logs, measurable lift, latency SLO, and rollback
Scale	Two-tower ANN; multiple retrievers; fusion/provenance; feature store; temporal datasets; model registry	Stable recall, point-in-time correctness, reproducible training
Quality	Multi-task ranker; session features; hard negatives; calibration; slate diversity; nearline features	Incremental online value without guardrail regression
Mature	Counterfactual evaluation; contextual exploration; advanced sequence/multimodal models; learned value/slate policy	Validated support/propensities, governance, and sustainable cost

What not to build first

- A giant Transformer before event contracts and baselines are trustworthy.
- Bandit optimization without randomized support and logged action propensities.
- A complex slate learner before candidate quality and duplicate handling are stable.
- Real-time everything; freshness should match signal velocity and user-value gain.
- Dozens of retrieval channels without attribution, quotas, or failure isolation.

Architecture should earn complexity incrementally. Each phase adds a capability only after instrumentation can measure its value and operations can support it.

Principal-level trade-offs

Decision	Benefit	Cost / risk	Decision rule
Multiple retrievers	Coverage and robustness	Fusion, infra, attribution	Add only for incremental union recall and online lift
Transformer sequence model	Richer short-term intent	Latency, memory, operations	Require incremental gain over summary/GRU baseline
CTR objective	Dense, responsive label	Clickbait and shallow engagement	Use as one calibrated task, never the sole utility
More real-time features	Faster adaptation	Cost, skew, cache loss	Match freshness to signal half-life and measured lift
More exploration	Learning and discovery	Immediate reward and user risk	Bound by cohort, safety, uncertainty, and guardrails
Personalization	Higher immediate relevance	Filter bubbles and privacy	Balance with diversity, controls, and minimization
Large ANN recall	Better candidate coverage	Memory and tail latency	Tune against end-to-end value, not recall alone
Offline optimization	Fast, cheap iteration	Policy bias and metric mismatch	Use for screening; require online causal evidence

How to communicate the design

State assumptions, identify the irreversible retrieval bottleneck, draw the online and learning paths, define stage budgets, explain the objective and bias, then close with failure modes, measurement, and evolution. A good design is a set of justified decisions, not a catalog of algorithms.

One-line takeaway: a video recommender is a hierarchical, multi-objective, feedback-driven decision system - not a single recommendation model.

Mental model and design checklist

Verb	Question
RETRIEVE	Did valuable content enter the funnel under extreme scale?
RANK	Can we estimate multiple candidate-level outcomes accurately?
VALUE	Are predictions calibrated and combined into durable user value?
RE-RANK	Is the slate valid, safe, diverse, fresh, and non-redundant?
EXPLORE	Are we learning beyond the incumbent policy with controlled risk?
LEARN	Are exposure, features, labels, experiments, and lineage trustworthy?
OPERATE	Can the system meet latency/cost SLOs and degrade safely?

Problem and architecture

- Clarify Home vs Up Next.
- State objective, scale, latency, freshness, and availability.
- Separate retrieval, ranking, value, and policy.
- Explain content ingestion and hard eligibility.
- Give candidate and latency budgets.
- Describe graceful fallback.

ML and data

- Define labels and negative sources.
- Explain long-term vs session intent.
- Use temporal, point-in-time-correct datasets.
- Address exposure/position/popularity bias.
- Define exploration and cold-start paths.

Evaluation and operations

- Retrieval, ranking, calibration, slate, product, and system metrics.
- A/B design and guardrails.
- Feature/index/model version compatibility.
- Drift, data quality, and source-contribution monitoring.
- Safety/privacy governance and deletion propagation.
- MVP-to-mature evolution with decision gates.

Final self-check

- What happens if the right video is not retrieved?
- What makes an unclicked impression a valid negative?
- Which assumptions are company claims vs design choices?
- Which feature must be real-time?
- Which dependency can fail without breaking playback?

Technical grounding and public-use note

- [1] Covington, Adams, and Sargin. **Deep Neural Networks for YouTube Recommendations**. RecSys 2016. doi.org/10.1145/2959100.2959190
- [2] Zhao et al. **Recommending What Video to Watch Next: A Multitask Ranking System**. RecSys 2019. doi.org/10.1145/3298689.3346997
- [3] Yi et al. **Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations**. RecSys 2019. [research.google publication](https://research.google.com/publication)
- [4] Ma et al. **Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts**. KDD 2018. [research.google publication](https://research.google.com/publication)
- [5] Kang and McAuley. **Self-Attentive Sequential Recommendation**. ICDM 2018. [arXiv:1808.09781](https://arxiv.org/abs/1808.09781)
- [6] Joachims, Swaminathan, and Schnabel. **Unbiased Learning-to-Rank with Biased Feedback**. WSDM 2017. [arXiv:1608.04468](https://arxiv.org/abs/1608.04468)
- [7] Li et al. **A Contextual-Bandit Approach to Personalized News Article Recommendation**. WWW 2010. [arXiv:1003.0146](https://arxiv.org/abs/1003.0146)
- [8] Johnson, Douze, and Jegou. **Billion-scale Similarity Search with GPUs**. 2017. [arXiv:1702.08734](https://arxiv.org/abs/1702.08734)
- [9] Malkov and Yashunin. **Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs**. [arXiv:1603.09320](https://arxiv.org/abs/1603.09320)

Scope note: this document is an educational system-design synthesis. Named platforms frame the problem; catalog sizes, candidate counts, and latency budgets are illustrative assumptions unless a citation explicitly states otherwise.

What was consolidated and corrected

The two Day 1 source PDFs were reconciled into one sequence. Repeated material was removed; missing production concerns were added, including content ingestion, point-in-time joins, propensity logging, ANN lifecycle, source-score normalization, false negatives, hard policy constraints, experiment mechanics, governance, and version compatibility. The original conceptual strengths - multi-stage retrieval, multi-task ranking, feedback bias, exploration, slate optimization, latency, fallback, and monitoring - were preserved.

END OF DAY 01

**Build the funnel. Measure every boundary.
Learn without losing control.**